

自然语言处理 (2025 春季学期)

十一、大语言模型的适配



大纲

- 基于提示的推断
- 多任务指令微调
- 基于人类反馈的强化学习
- 参数高效精调
- 大语言模型的中文适配
- 大语言模型压缩



基于提示的推断 (Prompt-based Inference)

□ 大语言模型是通用任务解决器

- 通过构建特定任务的自然语言提示，可以使大语言模型生成符合目标任务的输出，而不需要对模型进行微调
- 由于模型的自回归特性，模型的预测很大程度上依赖于“提示”的设计
- 为了获得理想的性能，需要对“提示”进行优化，这一过程称为“提示工程” (Prompt Engineering)

□ 提示 (Prompt) 通常包含以下要素

- 指令 (Instruction)：对于特定任务的描述
- 上下文 (Context)：必要的外部信息或额外的上下文信息
- 输入数据 (Input)：用户输入的内容或问题
- 输出指示 (Output)：指定输出的类型或格式



提示工程（Prompt Engineering）

□ 提示工程的通用技巧

- 清晰且具体的任务描述
- 提供必要的背景信息
- 明确对模型输出的期望
- 不要告诉模型“不要做什么”，而是告诉模型“要做什么”

好的提示	差的提示	分析
请提取以下文章中提到的所有公司名称及其关联的国家。将它们列在一个包含两列的表格中：公司名称和国家	提取文章中的公司	好的提示明确了需要提取的信息和格式
分析 XYZ 公司 2023 年第一季度到第三季度的销售数据，重点关注不同地区的增长趋势。请将结果总结为一份结构化的报告，突出主要增长区域和下降趋势	销售数据说明了什么	好的提示具体指明了需要分析的时间范围、公司和要关注的趋势
写一篇关于宋元战争时期的江湖侠义的短篇故事。该故事需包括一个出人意料的结尾，且字数应在 300~500 字	写一篇关于江湖侠义的故事	好的提示明确了故事的情节、背景、字数范围和需要的结尾
阅读以下客户评论文本。判断情感是积极、消极还是中立。提供你的分类并附上一句话解释理由	这个评论好还是不好	好的提示明确了需要进行的情感分析和需要提供的解释

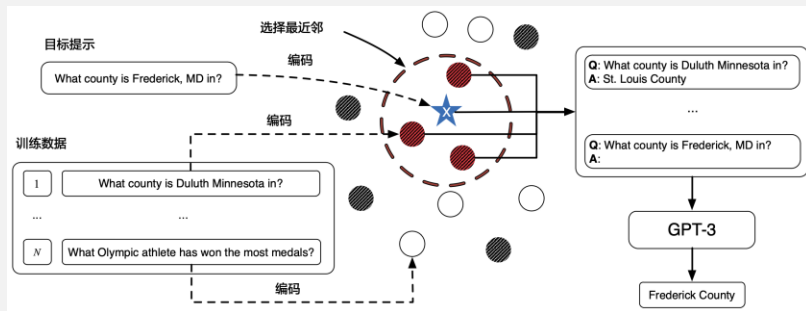




示例样本选择

□ 选择与测试样本相似的示例样本

— 基于检索的示例样本选择



<https://arxiv.org/pdf/2101.06804>

— 选择多样化且具有代表性的示例样本

— 随机排序示例样本，以避免引入近期偏置 (Recency Bias)



复杂问题分解

□ 思维链 (Chain-of-Thought)

— 在示例样本加入推理过程

<https://arxiv.org/pdf/2201.11903>

标准提示	思维链提示
模型输入 Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now? A: The answer is 11. Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?	模型输入 Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now? A: Roger started with 5 balls. 2 cans of 3 tennis balls each is 6 tennis balls. $5 + 6 = 11$. The answer is 11. Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?
模型输出 A: The answer is 27. ❌	模型输出 A: The cafeteria had 23 apples originally. They used 20 to make lunch. So they had $23 - 20 = 3$. They bought 6 more apples, so they have $3 + 6 = 9$. The answer is 9. ✅

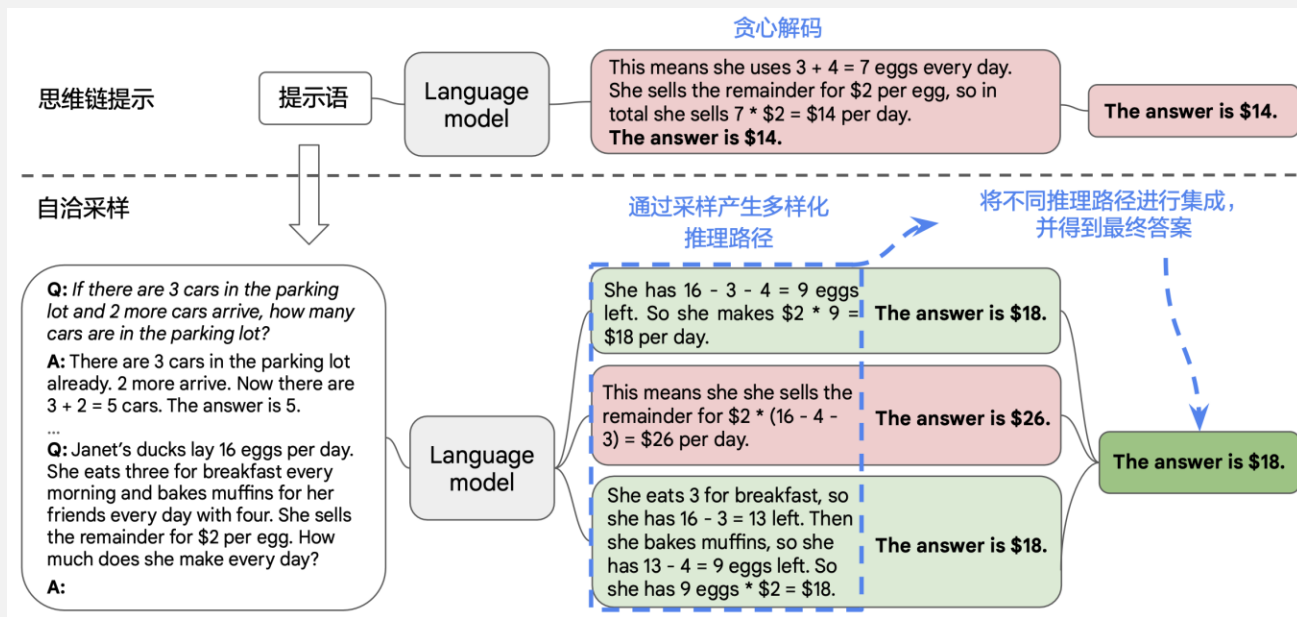
— 零样本情况下，可通过特定的提示来引导模型生成推理过程，如“Let's think step by step”





自洽采样 (Self-Consistency Sampling)

- 在思维链的基础上，模型通过“集成”多个不同推理链的预测结果来确定最优解



<https://arxiv.org/pdf/2203.11171>



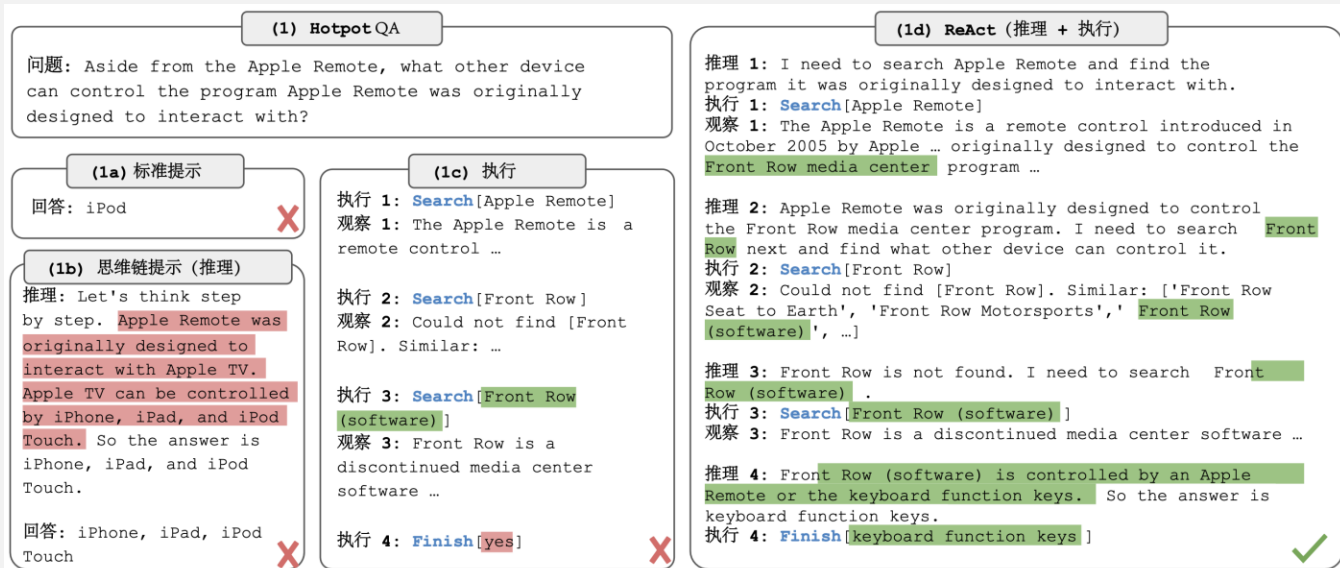
南京大学
NANJING UNIVERSITY



ReAcT: 推理与执行相结合

□ 在推理过程中，通过“执行”工具（如：搜索）从环境中获取反馈或者中间结果

— 适用于需要多步顺序推理的任务，如多跳式问题（Multi-hop QA）



<https://arxiv.org/pdf/2210.03629>



南京大学
NANJING UNIVERSITY

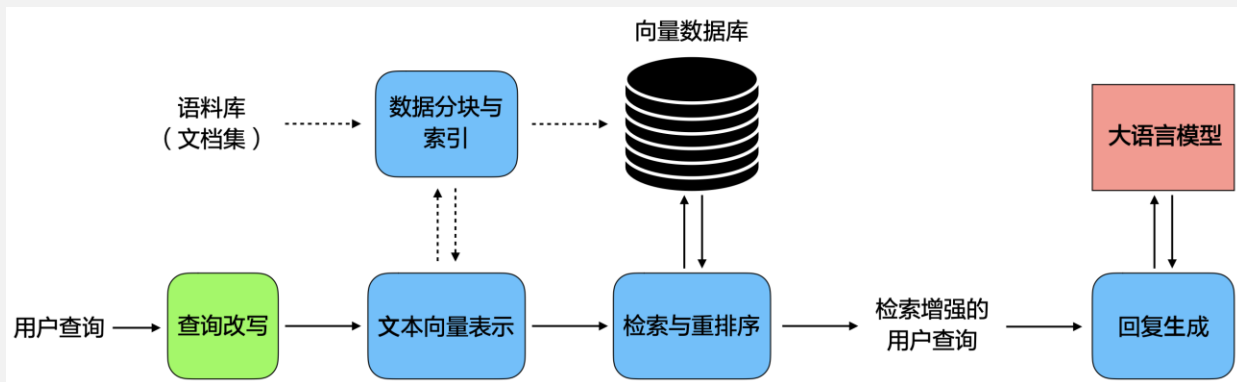


检索与工具增强

□ 大语言模型在下游应用中的局限

- 知识的全面性。模型的知识库无法涵盖所有领域的知识
- 知识的时效性。模型所依赖的知识库可能已过时，无法提供最新的信息
- 知识的准确性。模型的知识库中可能存在错误，无法完全保证信息的准确性

□ 检索增强生成（Retrieval-Augmented Generation）





大纲

- 基于提示的推断
- 多任务指令微调
- 基于人类反馈的强化学习
- 参数高效精调
- 大语言模型的中文适配
- 大语言模型压缩



多任务指令微调

□ 将大语言模型向人类期望对齐：多任务指令精调（Instruction Tuning）

- 将各种任务形式进行统一（指令+输入→输出）
- 在众多（成千上万）任务的标注数据上精调语言模型
- 模型能够处理未见任务（Zero-shot）

帮我总结下面这段文字：最大的遗憾就是被智子蒙住了眼睛，其实，豁达些想想：我们探索规律，与规律有何相干？也许有一天，人类或其他什么东西把规律探知到这种程度，不但能够用来改变他们自己的现实，甚至能够改变整个宇宙。

翻译这句话到英文：今天星期几

计算这道数学题：
 $567 * (9 + 1) =$

回答问题：什么是BERT



尽管探索规律会带来一些限制，但我们应该保持开放的心态去面对未知的可能性。

Today is what day?

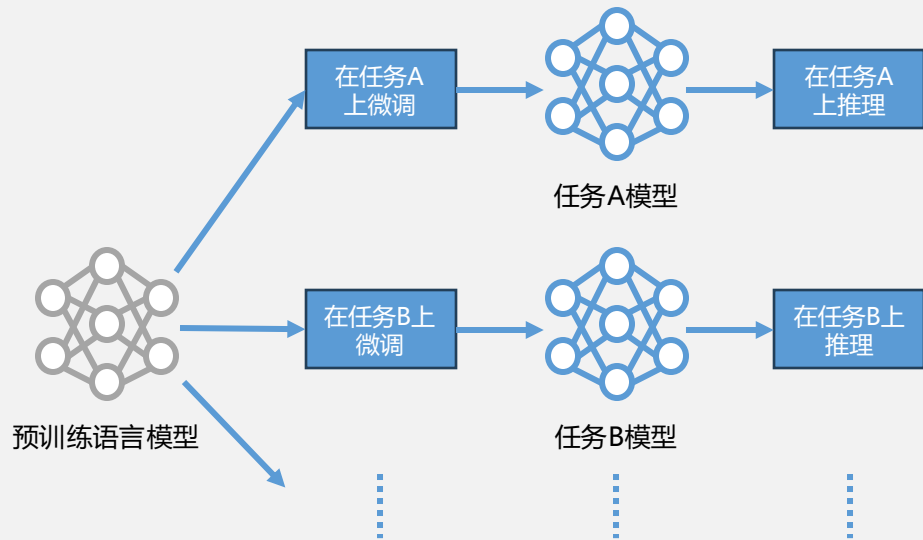
答案是 4576.

BERT (Bidirectional Encoder Representations from Transformers) 是一个预训练的自然语言处理的深度学习模型，由Google AI Language团队于2018年发布。它是一种基于Transformer模型的双向编码器，可以用于许多NLP任务，如文本分类、问答、命名实体识别等……





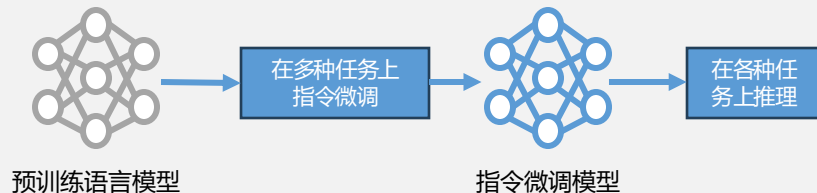
传统微调、提示学习与指令微调的对比



(a) 传统微调



(b) 提示学习



(c) 指令微调





指令数据格式

□ Instruction

- 描述模型需要执行的指令内容

□ Input（可选）

- 任务上下文或输入信息，例如当指令是“对文章进行总结”，则input是文章内容

□ Output

- 由text-davinci-003生成的针对指令的回复

Below is an instruction that describes a task, paired with an input that provides further context. Write a response that appropriately completes the request.

Instruction:

{instruction}

Input:

{input}

Response:

▲ 包含“input”字段的指令模板

Below is an instruction that describes a task, paired with an input that provides further context. Write a response that appropriately completes the request.

Instruction:

{instruction}

Response:

▲ 不含“input”字段的指令模板





代表性指令微调数据集

数据集	多轮对话	语言	构建方式	版权情况
Alpaca	否	英文	自动	非商用
ShareGPT	是	英文	半自动	非商用
Baize	是	英文	自动	非商用
Dolly	否	英文	人工	可商用
OpenAssistant	是	多语言	人工	可商用



指令微调的优缺点

□ 优点

- 方法简单直接
- 能泛化到未见任务

□ 缺点

- 数据收集代价高昂
- 语言模型的目标和人类的偏好不匹配
 - 很多任务没有标准答案，如对话、文本生成
 - 语言模型等价看待每个token的错误





大纲

- 基于提示的推断
- 多任务指令微调
- 基于人类反馈的强化学习
- 参数高效精调
- 大语言模型的中文适配
- 大语言模型压缩

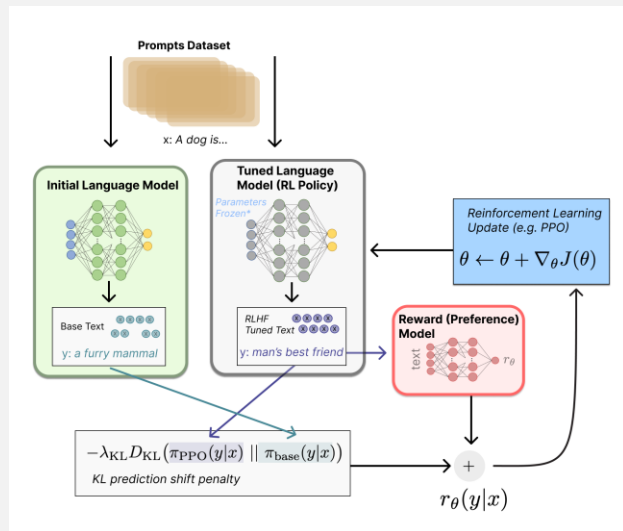
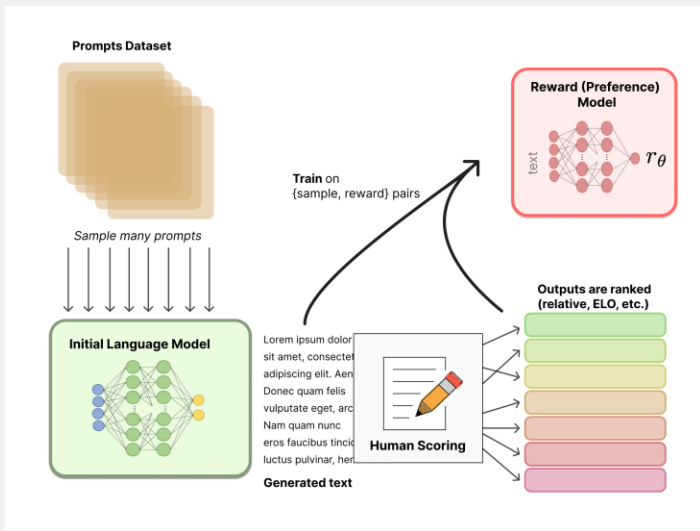




人类反馈强化学习

□ RLHF (Reinforcement Learning from Human Feedback)

— 奖励模型 + 强化学习





奖励模型训练

□ 训练奖励模型学习人类偏好

- 标注数据更容易获取
- 比较同一个prompt的不同输出，并进行打分

□ 给定由一个prompt生成的K个输出，标注人员需要对 C_K^2 个输出排序

训练方式

$$\text{loss}(\theta) = -\frac{1}{\binom{K}{2}} E_{(x, y_w, y_l) \sim D} [\log(\sigma(r_\theta(x, y_w) - r_\theta(x, y_l)))]$$

一对训练数据

奖励模型打分

<https://arxiv.org/abs/2203.02155>



南京大学
NANJING UNIVERSITY



Proximal Policy Optimization (PPO)

- ❑ 从数据集中sample一个prompt: x
- ❑ 语言模型（policy）根据 x 生成输出 y
- ❑ 使用奖励模型（Environment）计算得分 $r_{\theta}(x,y)$ （Reward）
- ❑ 由 $r_{\theta}(x,y)$ 使用PPO算法优化语言模型

奖励模型对 (x,y) 的打分

KL奖励系数, 论文中 $\beta = 0.02$

$$\text{objective}(\phi) = E_{(x,y) \sim D_{\pi_{\phi}^{\text{RL}}}} [r_{\theta}(x,y) - \beta \log(\pi_{\phi}^{\text{RL}}(y|x) / \pi^{\text{SFT}}(y|x))] +$$

第一步的SFT模型

$$\gamma E_{x \sim D_{\text{pretrain}}} [\log(\pi_{\phi}^{\text{RL}}(x))]$$

预训练loss系数
PPO: $\gamma = 0$; PPO-ptx: $\gamma \neq 0$
论文中 $\gamma = 27.8$

正在进行强化训练的模型,
即最终的InstructGPT, 初始
化时等于SFT模型

<https://arxiv.org/abs/2203.02155>



南京大学
NANJING UNIVERSITY



RLHF的优缺点

□ 优点

- 降低人工标注难度
- 进一步将大语言模型向人类期望对齐
- 结果更符合人类的预期（多样性、安全性）
- 利用真实用户的反馈（AI正循环）

□ 缺点

- 步骤繁琐
- RL模型不容易训练
- 需要更多的计算资源





大纲

- 基于提示的推断
- 多任务指令微调
- 基于人类反馈的强化学习
- 参数高效微调
- 大语言模型的中文适配
- 大语言模型压缩





参数高效微调

□ LoRA (Low-rank adaptation)

- 问题背景：大模型参数量大，微调代价高；
- 核心思想：冻结原始参数，仅训练一个低秩矩阵对权重增量进行建模；
- 优势：在保证精度的前提下显著减少训练参数量和内存开销。

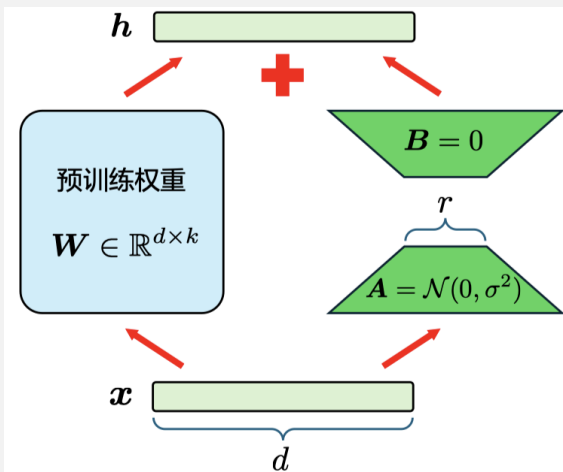


图 9-12 LoRA 的基本流程

$$h = Wx + \Delta Wx = Wx + \frac{\alpha}{r}BAx$$

$$\Delta W = BA$$



参数高效微调

□ QLoRA

- 背景问题：LoRA 仍难精调超大模型（如 Llama 65B）
- 核心思路：将基模型参数量化为 4 位，只训练少量 BF16 的 LoRA 权重
- 关键技术：4位NormalFloat、双重量化、分页优化器

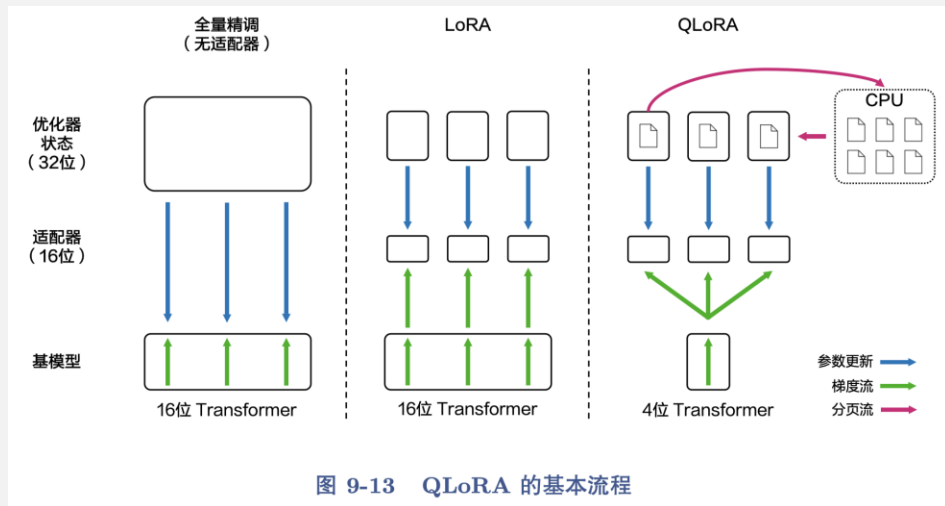


图 9-13 QLoRA 的基本流程



参数高效微调

□ QLoRA

- 4位 NormalFloat
 - 假设模型参数服从零均值高斯分布；
 - 使用 4 位分位量化编码映射至 $[-1, 1]$ ；
 - 比传统 4-bit 整型在误差控制上表现更优。
- 双重量化（Double Quantization）
 - 对块级量化中的量化常数再次量化；
 - 减少块级常数的存储成本；
 - 例如块大小 64 时，平均开销从 0.5 降至 0.127 bit/param。
- 分页优化器（Paged Optimizer）
 - 基于 CUDA 统一内存管理；
 - 自动迁移优化器状态，缓解显存瓶颈；
 - 有效防止训练中内存溢出。





参数高效微调

□ Adapter

— 核心思路

- 在 Transformer 各层插入小型瓶颈模块，仅训练少量参数；
- 保持主干模型冻结，支持 多任务切换；
- 避免重复训练整个模型，适用于资源受限和任务切换频繁场景。

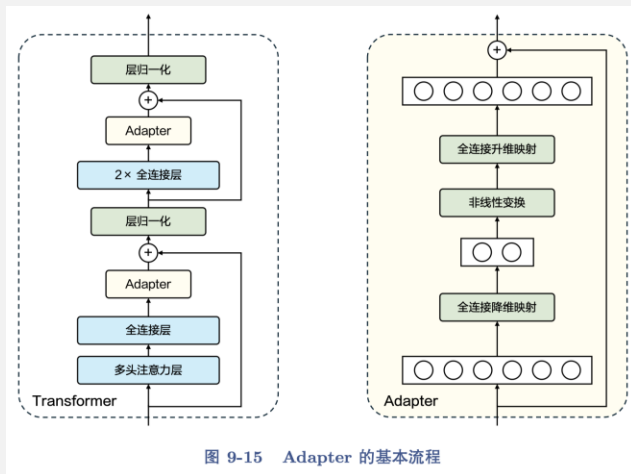


图 9-15 Adapter 的基本流程





参数高效微调

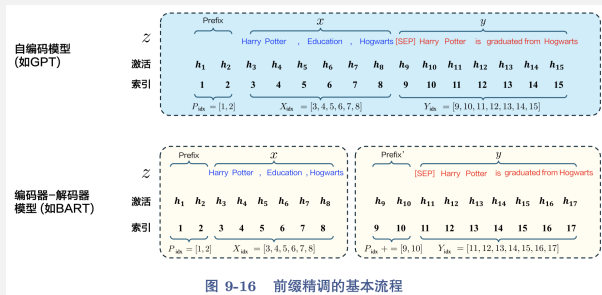
□ Prefix-tuning

— 方法动机

- 来源于 Prompting 思路，在输入前添加特定上下文能引导语言模型输出；
- 相比手工设计指令，Prefix-Tuning 采用可训练的“软前缀”，更灵活、效果更优；
- 模型主干参数全冻结，仅训练前缀参数，节省计算资源。

— 核心机制

- 在输入前拼接一个 $N_p \times d$ 的可训练前缀 P ， N_p 为前缀长度， d 为隐藏层维度；
- 前缀表示为“虚拟 token 向量”，不对应实际词汇表项，但能表达抽象语义；
- 每层 Transformer 都插入前缀参数，参与上下文构建；
- 训练时，仅更新前缀 P ，主模型保持不变。





参数高效微调

□ Prefix-tuning

— 方法动机

- 来自自编码模型 (如GPT)
- 相
- 模

— 核心机制

- 在
 - 前
 - 每
 - 训
- 编码器-解码器模型 (如BART)

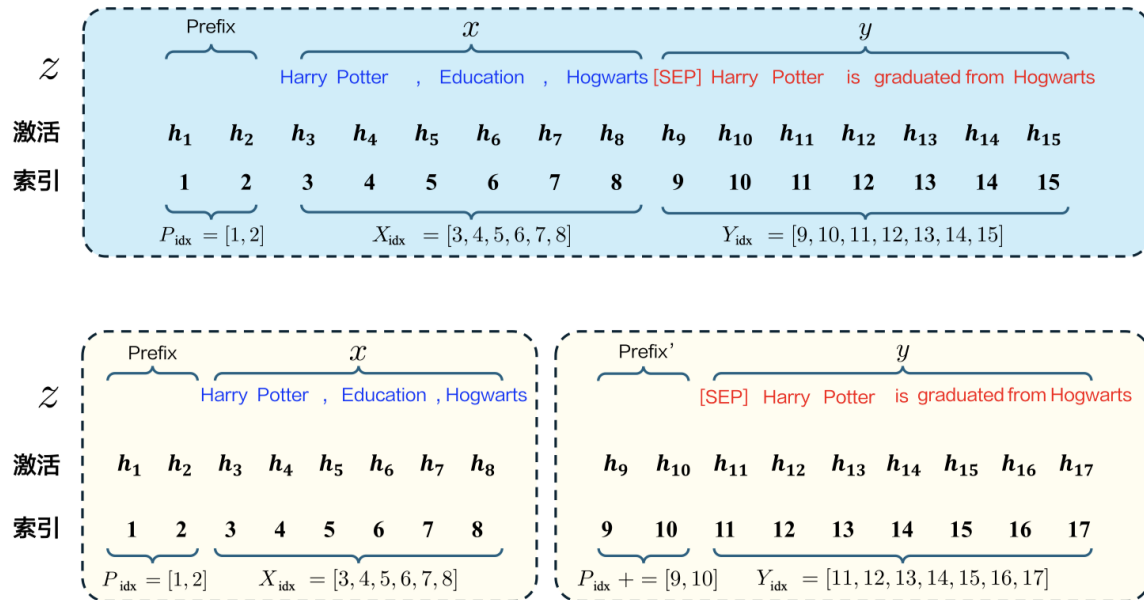


图 9-16 前缀精调的基本流程



参数高效微调

□ P-tuning

— 应用场景

- 相较于 Prefix-Tuning 主要用于生成任务，P-tuning 更适用于自然语言理解任务，如文本分类、序列标注、阅读理解等
- 本质是对输入序列进行结构化改写，插入可训练的“虚拟提示”

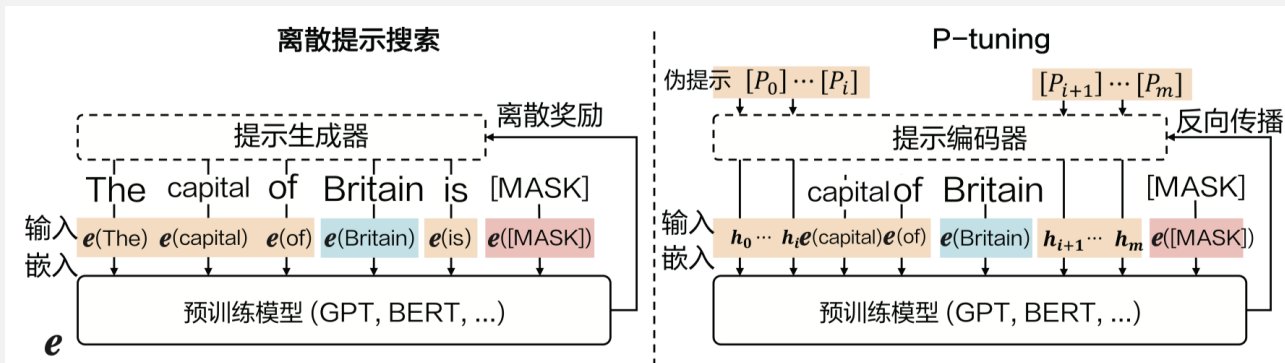


图 9-17 模式精调的基本流程

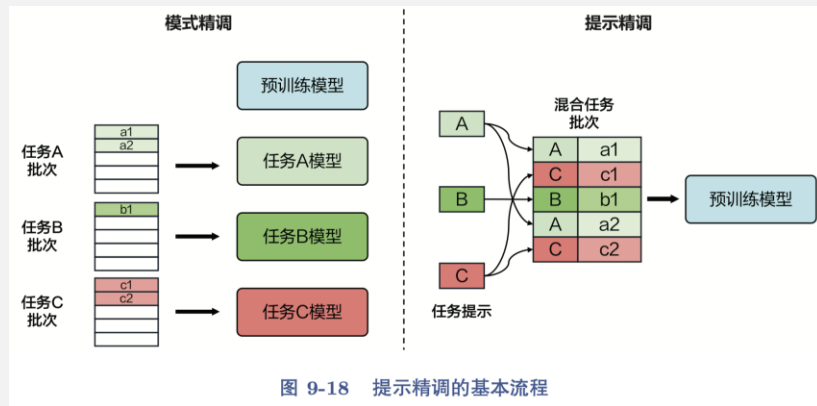


参数高效微调

□ Prompt-tuning

- 与前缀精调、模式精调类似，Prompt-tuning 使用可训练“软提示”代替人工提示；

- 输入序列 $X = \{x_1, x_2, \dots, x_n\}$ ，嵌入后为 $e \in \mathbb{R}^{n \times d}$
- 添加一组可训练的软提示权重 $p \in \mathbb{R}^{m \times d}$ ，拼接为 $[p; e] \in \mathbb{R}^{(n+m) \times d}$
- 仅更新软提示 p ，模型主体权重 保持冻结





大纲

- 基于提示的推断
- 多任务指令微调
- 基于人类反馈的强化学习
- 参数高效微调
- 大语言模型的中文适配
- 大语言模型压缩



大语言模型的中文适配

□ 动机：LLaMA没有显式使用中文语料进行训练

- 原版LLaMA没有显式使用中文语料进行训练，词表（32K tokens）中仅包含非常少量的中文字符
- 经粗略统计，LLaMA词表中仅包含约700个中文字符（范围：\u4E00-\u9FFF）

['的','大','中','一','人','子','日','山','文','本','数','国','新','是','田','道','村','用','天','镇','不','学','有','上','会','小','行','三','市','王','区','年','生','成','定','个','和','在','方','高','地','月','公','出','法','名','下','社','为','乡','川','之','字','海','东','明','时','南','西','平','式','家','第','城','美','件','部','女','神','以','太','前','金','野','北','加','原','置','安','我','京','可','取','县','二','理','自','信','代','要','化','木','水','島','正','内','作','合','回','了','所','事','表','分','入','全','里','郎','据','州','者','通','都','江','到','对','使','世','果','十','藤','来','面','这','街','石','能','空','武','后','元','河','町','花','类','物','版','动','如','真','号','林','書','民','口','示','图','戰','李','光','白','心','设','路','最','手','型','建','主','集','目','度','长','星','五','重','于','发','史','國','你','駅','多','士','四','司','次','立','点','音','器','井','存','当','寺','性','也','位','值','古','院','界','語','愛','時','东','同','保','青','体','清','相','情','将','族','宮','政','间','松','始','息','少','教','获','列','开','科','春','治','吉','台','工','知','八','場','画','百','記','得','氏','关','标','外','間','校','制','力','門','好','指','色','返','馬','请','風','接','志','魔','更','程','郡','利','県','周','谷','香','期','初','福','片','動','參','義','象','功','过','皇','特','长','英','其','流','除','永','直','千','館','朝','单','格','德','歌','进','限','夫','園','量','土','放','码','等','系','華','常','否','見','源','实','博','解','男','無','查','条','後','他','网','阿','户','命','就','龍','君','夏','言','先','典','府','组','改','华','与','调','由','修','學','消','符','录','已','陽','容','未','宗','龙','提','六','形','伊','火','佐','结','九','雄','而','张','向','选','仁','堂','久','义','无','那','線','务','基','属','配','軍','津','完','研','注','失','应','友','章','求','们','模','需','電','此','夜','或','橋','根','玉','交','品','良','则','開','被','株','记','會','经','转','崎','比','造','没','现','七','商','机','阳','角','站','及','術','认','创','編','伝','岡','港','任','登','布','究','帝','密','变','序','计','曲','传','包','意','去','沙','写','超','今','森','黄','景','湖','何','宇','張','语','老','例','鉄','克','然','報','服','想','実','载','波','马','状','线','洋','万','添','球','機','支','显','拉','送','隊','处','師','像','黑','只','起','段','區','選','業','算','广','视','秋','因','输','康','思','死','聖','头','車','按','夢','汉','从','題','展','省','葉','素','閨','页','共','宿','恣','技','乐','控','移','影','管','戸','函','尾','场','介','育','泉','说','換','必','紀','宝','気','门','令','左','漢','若','屋','局','打','発','问','恋','兵','別','并','节','引','统','智','电','赤','断','称','身','首','付','連','官','持','奈','御','親','库','秀','址','守','活','藏','竹','草','結','昌','樹','勝','足','航','陳','业','富','雪','再','默','楽','沢','羅','忠','錯','桥','雲','該','岩','專','切','店','朱','幸','母','串','击','設','經','座','云','告','变','试','隆','判','劉','编','达','右','复','看','話','坂','尔','衛','丸','样','鬼','喜','斯','銀','群','近','塔','确','索','非','望','希','甲','越','鳥','麻','雅','拳','溪','測','话','池','菜','食','渡','速','陈','健','军','庄','红','論','孝','頭','飛','么','達','巴','洞','貴','项','种','运','彦','书','构','米','连','操','装','反','飯','员','昭','兴','客','删','居','現','助','唐','瀬','微','舞','导','效','深','梅','料','每','洲','茶','败','些','双','嘉','進','丁','故','計','遠','候','房','兩','才','止','番','奇','怪','联','泰','边','还','黃','收','弘','给']



大语言模型的中文适配

□ 切词示例

- 由于LLaMA使用了sentencepiece分词器，对于不在词表中的文本会切分为byte-level 字符
- 好处：未登录词不会全部映射到同一个token，如<UNK>
- 坏处：切词结果不可读，未登录词的编解码效率显著降低
- 以下切词示例表明，通过扩充中文词表，中文编解码效率可翻倍

人工智能是计算机科学、心理学、哲学等学科融合交叉学科。

原版LLaMA分词
(32K词表)



‘，’，‘人’，‘工’，‘智’，‘能’，‘是’，‘计’，‘算’，‘机’，‘科’，‘学’，‘，’，‘，’，‘心’，‘理’，‘学’，‘，’，‘，’，‘0xE5’，‘0x93’，‘0xB2’，‘学’，‘等’，‘学’，‘科’，‘0xE8’，‘0x9E’，‘0x8D’，‘合’，‘的’，‘交’，‘0xE5’，‘0x8F’，‘0x89’，‘学’，‘科’，‘。’

35个token > 16个token



中文LLaMA分词
(50K词表)



‘，’，‘人工智能’，‘是’，‘计’，‘算’，‘机’，‘科学’，‘，’，‘，’，‘心理’，‘学’，‘，’，‘，’，‘哲学’，‘等’，‘学’，‘科’，‘融合’，‘的’，‘交叉’，‘学科’，‘。’



大语言模型的中文适配

□ 中文词表扩充方法

- 第一步：使用sentencepiece工具在中文预训练语料上训练出20K个单词的中文词表
- 第二步：删除上述20K中文词表中包含原版LLaMA词表（32K）的部分
- 第三步：在原版LLaMA词表上拼接去重后的中文词表，得到最终词表
- 一代中文LLaMA词表大小扩充至49953，二代中文LLaMA进一步扩充至55396

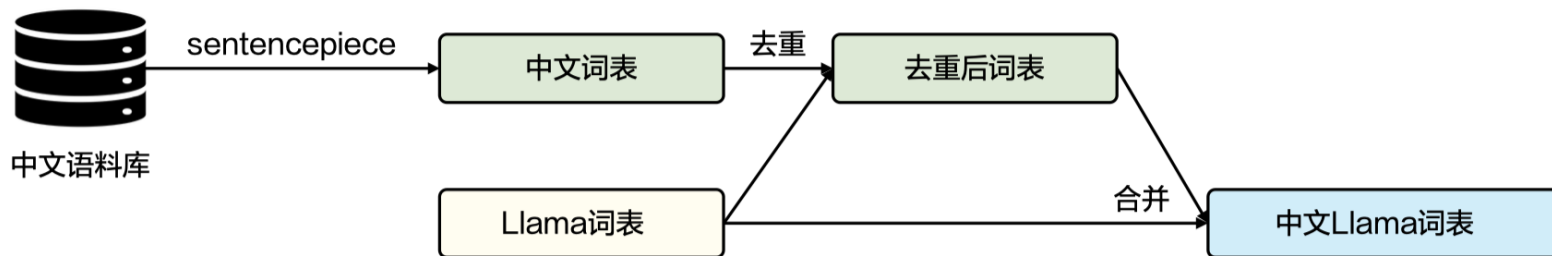


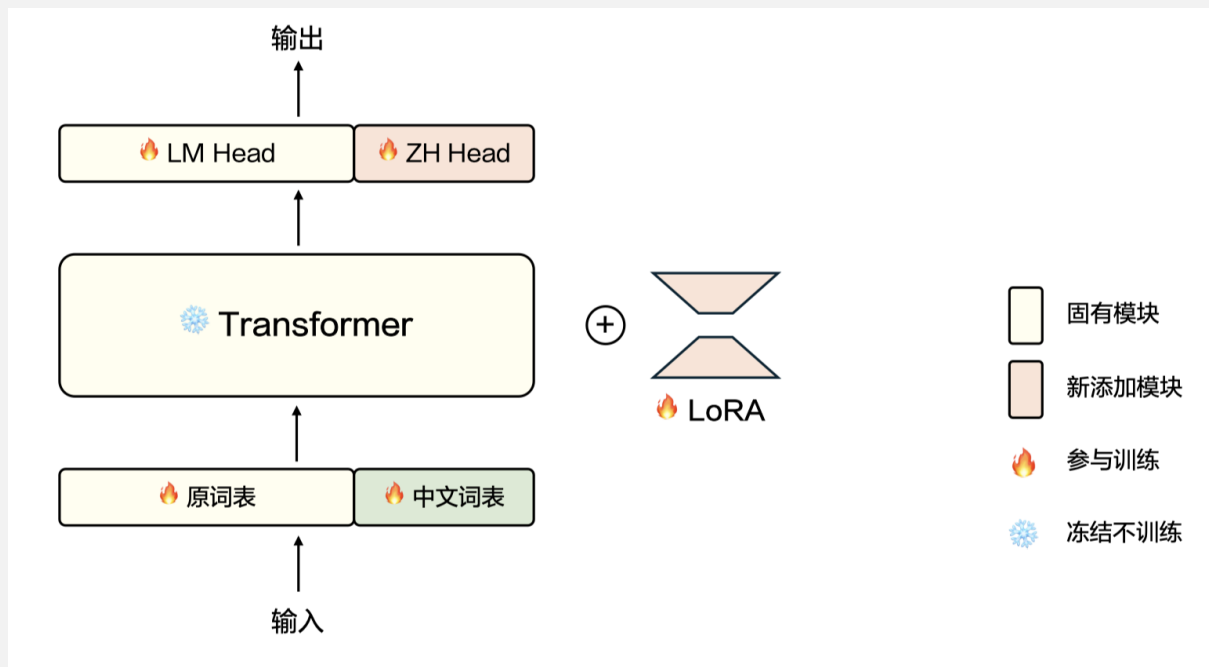
图 9-19 中文词表扩充的流程



大语言模型的中文适配

□ 中文增量训练

- 采用基于LoRA的高效训练方法，在保持英文能力的同时，注入中文知识





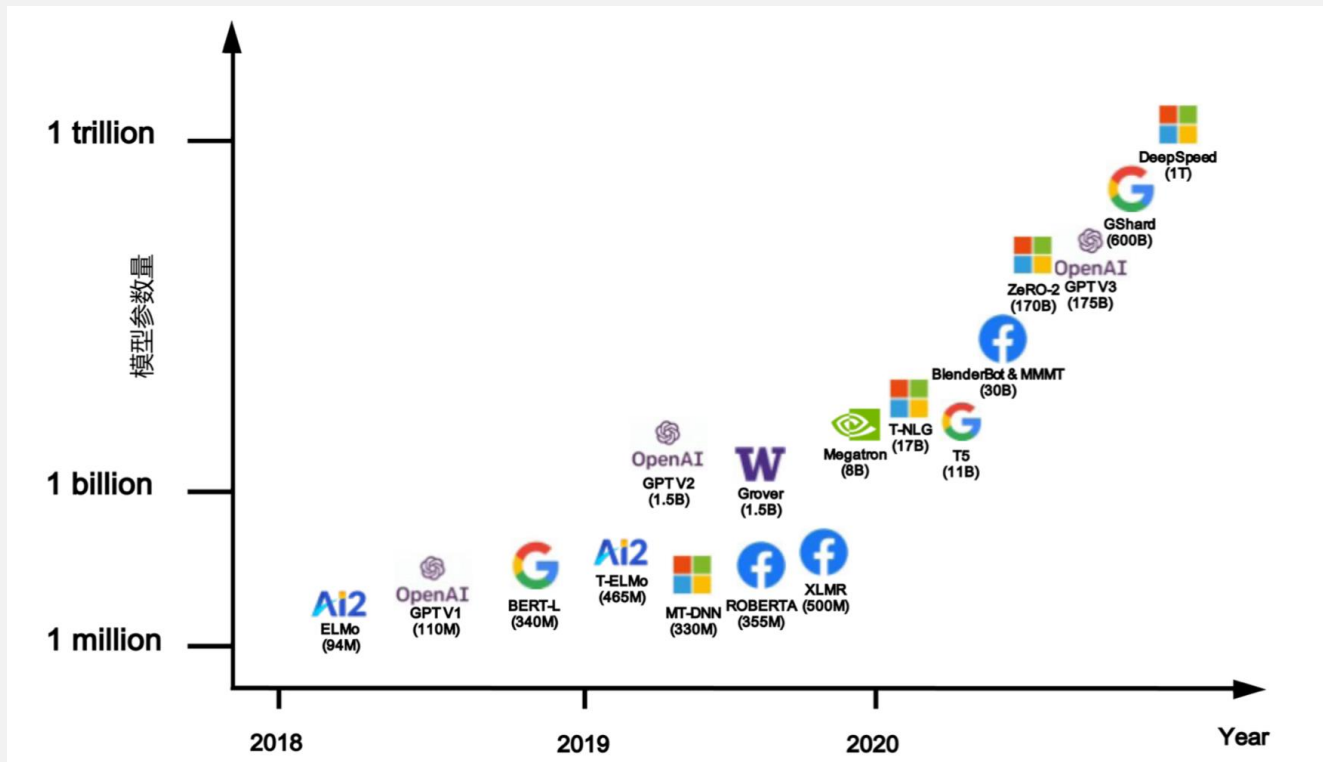
大纲

- 基于提示的推断
- 多任务指令微调
- 基于人类反馈的强化学习
- 参数高效微调
- 大语言模型的中文适配
- 大语言模型压缩



大语言模型的压缩

□ 预训练模型参数量趋势

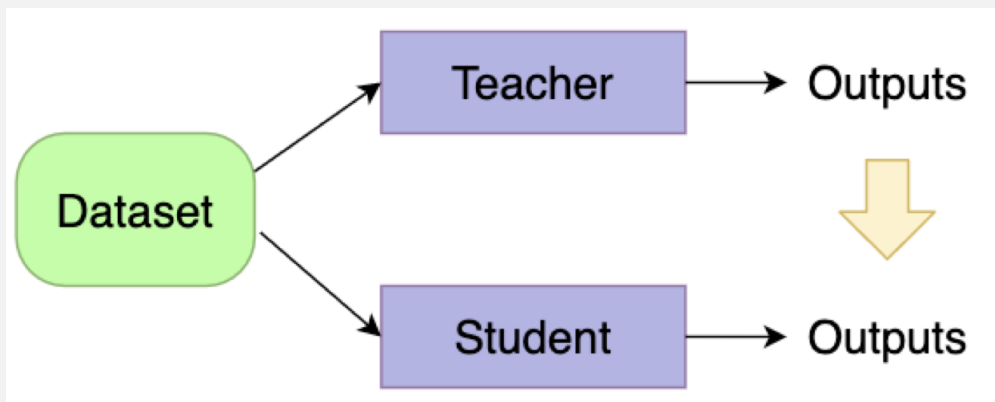




大语言模型的压缩

□ 为什么需要模型压缩与知识蒸馏技术？

- 预训练模型通常需要占用很大的空间，并且训练和推断时间也很慢
- 直接在实际产品或应用中使用预训练模型难以满足时间和空间需求
- 知识蒸馏技术可以在不损失或少量损失性能的情况下，将大模型的知识迁移到小模型，从而提升推断速度



知识蒸馏过程：“老师教学生”

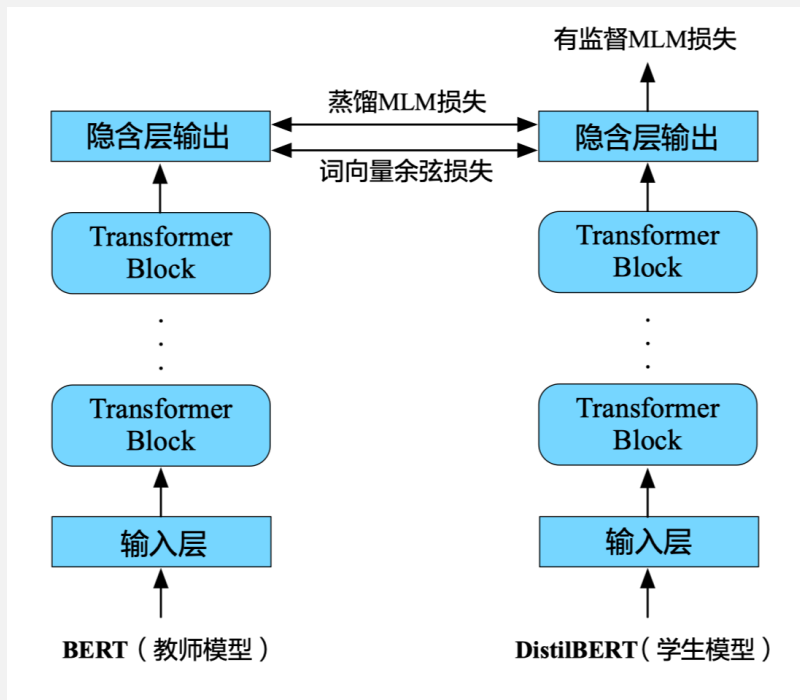




大语言模型的压缩

□ DistilBERT

- 通用、任务无关的6层BERT
- 相比BERT-base, 小40%、快60%、在NLU任务上可达到原模型的97%
- 使用MLM预训练任务进行知识蒸馏(无NSP)
- 训练目标由以下三部分组成
 - 由数据集自带的硬标签(hard-labels)计算的有监督MLM损失
 - 由教师模型提供的软标签(soft-labels)计算的蒸馏MLM损失
 - 教师模型和学生模型隐层输出之间的余弦相似度损失





大语言模型的压缩

□ DistilBERT

- 总损失函数

$$\mathcal{L} = \mathcal{L}^{\text{s-mlm}} + \mathcal{L}^{\text{d-mlm}} + \mathcal{L}^{\text{cos}}$$

- 有监督MLM损失

$$\mathcal{L}^{\text{s-mlm}} = - \sum_i y_i \log(s_i)$$

- 蒸馏MLM损失

$$\mathcal{L}^{\text{d-mlm}} = - \sum_i t_i \log(s_i)$$

- 词向量余弦损失

$$\mathcal{L}^{\text{cos}} = \cos(\mathbf{h}^{\text{t}}, \mathbf{h}^{\text{s}})$$



大语言模型的压缩

□ TextBrewer: An Open-Source Knowledge Distillation Toolkit

- 推出了首个面向自然语言处理领域的基于Pytorch的知识蒸馏工具包
- 提供了方便、快捷、易用的知识蒸馏框架，少量性能损失换取大幅速度提升
- <http://textbrewer.hfl-rc.com/> 或通过 `pip install textbrewer` 安装



TextBrewer

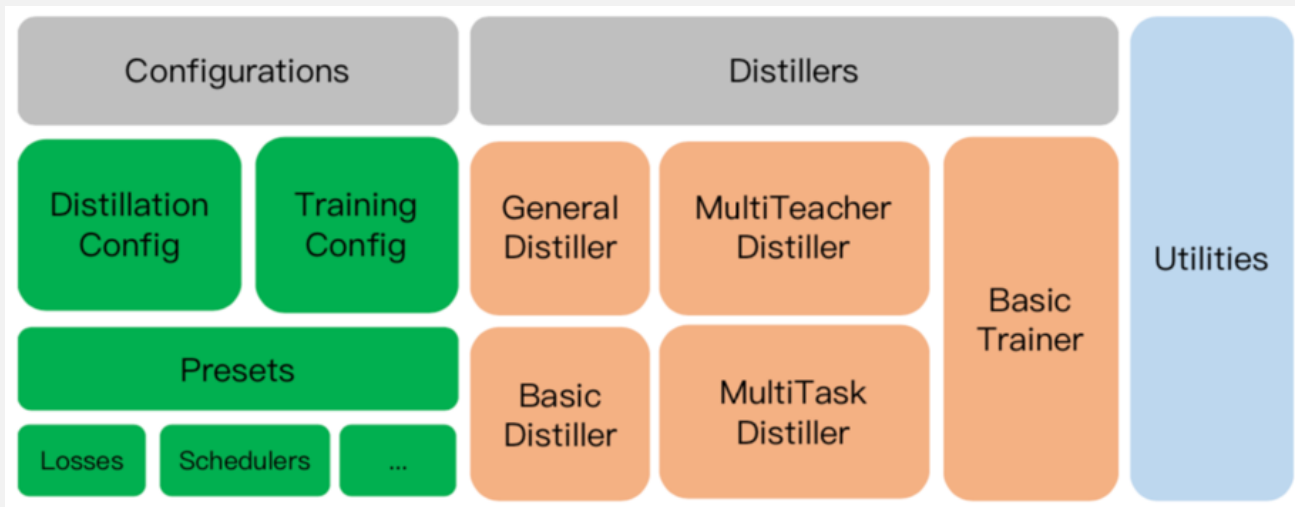
- 模型无关：适用于多种模型结构（主要面向Transformer结构）
- 方便灵活：可自由组合多种蒸馏方法，支持增加自定义损失等模块
- 非侵入式：无需对教师与学生模型本身结构进行修改
- 适用面广：支持典型NLP任务，如文本分类、阅读理解、序列标注等



大语言模型的压缩

□ TextBrewer: 整体架构

- Distillers: 用于执行实际的知识蒸馏工作
- Configurations: 为Distillers提供必要的配置
- Utilities: 包含一些辅助的功能，如模型参数统计等



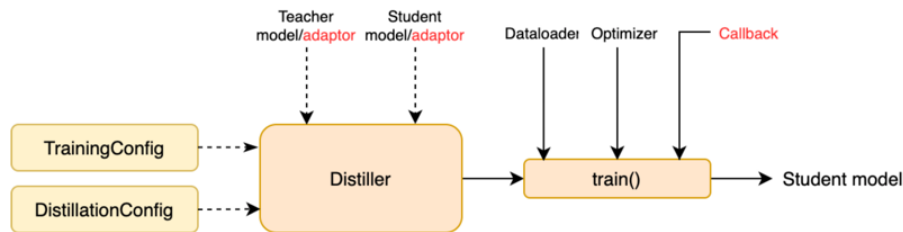


大语言模型的压缩

□ TextBrewer:工作流程

- 第一步：开始蒸馏之前的准备工作
 - 训练教师模型，定义并初始化学生模型
 - 构造蒸馏用数据集的DataLoader
- 第二步：知识蒸馏
 - 初始化Distiller，构造训练配置和蒸馏配置
 - 定义adaptors和callback，分别用于适配模型输入输出和训练过程中的回调
 - 调用Distiller的train方法开始蒸馏
- 一共只需20行左右的代码！

```
1 from textbrewer import GeneralDistiller
2 from textbrewer import TrainingConfig, DistillationConfig
3
4 # We omit the initialization of models, optimizer, and dataloader.
5 teacher_model : torch.nn.Module = ...
6 student_model : torch.nn.Module = ...
7 dataloader : torch.utils.data.DataLoader = ...
8 optimizer : torch.optim.Optimizer = ...
9 scheduler : torch.optim.lr_scheduler = ...
10
11 def simple_adaptor(batch, model_outputs):
12     # We assume that the first element of model_outputs
13     # is the logits before softmax
14     return {'logits': model_outputs[0]}
15
16 train_config = TrainingConfig()
17 distill_config = DistillationConfig()
18 distiller = GeneralDistiller(
19     train_config=train_config, distill_config=distill_config,
20     model_T=teacher_model, model_S=student_model,
21     adaptor_T=simple_adaptor, adaptor_S=simple_adaptor)
22
23 distiller.train(optimizer, scheduler,
24                dataloader, num_epochs, callback=None)
```





大语言模型的压缩

□ TextBrewer: 实验效果

- 教师模型: **BERT-base (110M)**
- 学生模型: **T6 (60%), T3 (41%), T3-small (16%), T4-tiny (与TinyBERT相同大小, 13%)**
- 单教师知识蒸馏
 - T6结构可以达到教师模型效果的99%, 且模型体积缩小至60%
 - T4-tiny知识蒸馏结果优于TinyBERT
- 多教师知识蒸馏
 - 所有模型使用同样的结构, 即**BERT-base**
 - 蒸馏后的学生模型获得最优效果, 且超过简单的模型融合 (ensemble)

Model	MNLI		SQuAD		CoNLL-2003
	m	mm	EM	F1	F1
BERT _{BASE}	83.7	84.0	81.5	88.6	91.1
<i>Public</i>					
DistilBERT	81.6	81.1	79.1	86.9	-
TinyBERT	80.5	81.0	-	-	-
+DA	82.8	82.9	72.7	82.1	-
<i>TextBrewer</i>					
BiGRU	-	-	-	-	85.3
T6	83.6	84.0	80.8	88.1	90.7
T3	81.6	82.5	76.3	84.8	87.5
T3-small	81.3	81.7	72.3	81.4	78.6
T4-tiny	82.0	82.6	73.7	82.5	77.5

Model	MNLI		SQuAD		CoNLL-2003
	m	mm	EM	F1	F1
Teacher 1	83.6	84.0	81.1	88.6	91.2
Teacher 2	83.6	84.2	81.2	88.5	90.8
Teacher 3	83.7	83.8	81.2	88.7	91.3
Ensemble	84.3	84.7	82.3	89.4	91.5
Student	84.8	85.3	83.5	90.0	91.6



大语言模型的压缩

□ 模型裁剪

- 模型裁剪是指通过移除神经网络中的一部分权重或神经元 来减少模型的大小和计算需求的过程
- 非结构化裁剪
 - 涉及移除单个权重，这些权重根据某种标准被认为是不重要的
 - 非结构化裁剪的结果是一个稀疏的权重矩阵，可能需要专门的硬件 或软件优化来实现计算效率
- 结构化裁剪
 - 结构化裁剪按照预定义的网络结构（如神经元、卷积核或整个层）来移除权重
 - 结构化裁剪的结果是一个更紧凑的模型， 容易在标准硬件上实现效率提升，因为它减少了模型的维度

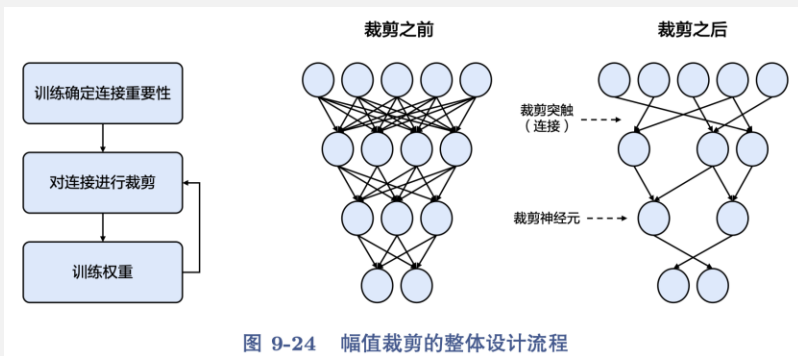


大语言模型的压缩

□ 模型裁剪

— 非结构化裁剪 —— 以幅值裁剪（Magnitude Pruning）为例

- 预训练: 正常训练网络, 以识别重要 vs. 不重要的权重连接; 目的不是获得最优性能, 而是判断权重贡献大小
- 权重剪除: 根据绝对值排序, 裁剪掉低于设定阈值的权重; 剩下的是“重要连接”
→ 得到稀疏结构
- 微调重训练: 对稀疏网络重新训练, 恢复模型性能; 如果跳过此步骤, 模型准确率将严重下降





大语言模型的压缩

□ 模型裁剪

– 结构化裁剪

- 结构化裁剪是一种以通道、注意力头、FFN 单元、甚至整个层为单位的压缩方法，目标是在尽量不影响模型性能的前提下减少计算量与参数规模。由于裁剪结果仍保持稠密结构，因此无需依赖稀疏计算支持，适用于通用硬件部署
- 裁剪标准通常基于：
 - 权重范数（L1 / L2）
 - 对损失函数的影响程度（如灵敏度分析）
 - 以注意力头为例，可通过以下方式评估重要性：

$$I_h = \mathbb{E}_{x \sim X} \left| \frac{\partial \mathcal{L}(x)}{\partial \xi_h} \right|$$



大语言模型的压缩

□ TextPruner: A Model Pruning Toolkit for Pre-trained Language Model

- 推出首个面向自然语言处理领域的基于PyTorch的模型裁剪工具包
- 通过轻量、快速的裁剪方法对模型进行结构化剪枝，从而实现压缩模型体积、提升模型速度
- 访问 <http://textpruner.hfl-rc.com/> 或通过 `pip install textpruner` 安装



TextPruner

- **功能通用**：适配多种预训练模型，以及多种NLU任务
- **可定制化**：除了标准PLM外，也可使用TextPruner裁剪基于标准PLM开发的自定义模型
- **灵活便捷**：可作为Python包在Python脚本中使用，也提供了独立的命令行工具
- **运行高效**：使用无训练的结构化裁剪方法，运行迅速，快于知识蒸馏等基于训练的方法



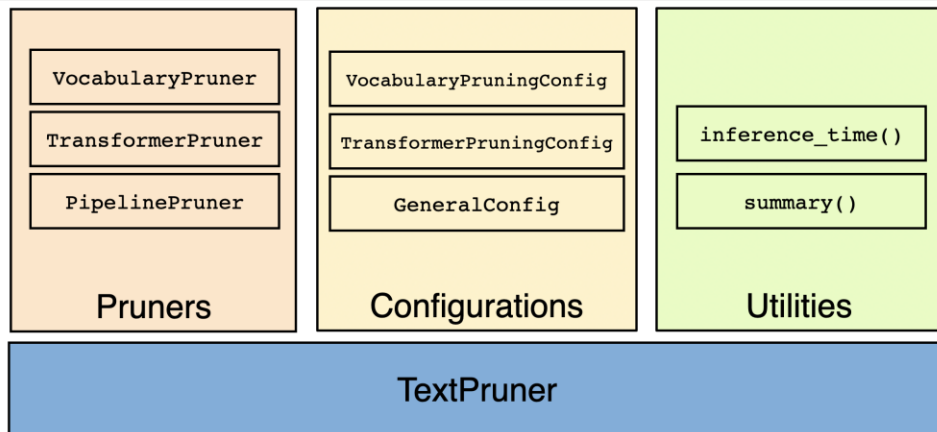


大语言模型的压缩

□ TextPruner

— 工具包设计架构

- **Pruners:** 用于执行实际的模型裁剪，包含词表裁剪、Transformer裁剪、流水线裁剪等
- **Configurations:** 为Distillers提供必要的配置信息
- **Utilities:** 包含一些辅助的功能，如模型参数统计、计算推断时间等



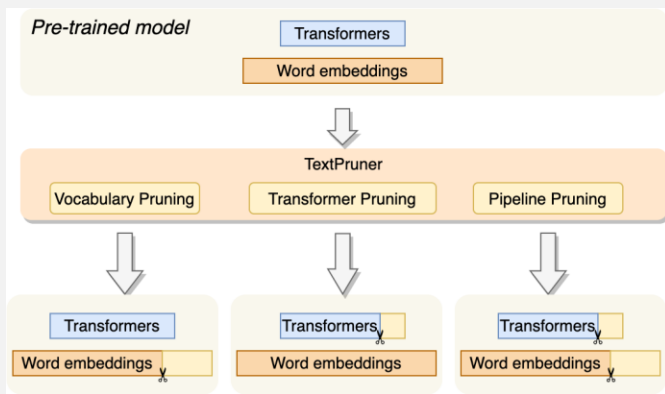


大语言模型的压缩

□ TextPruner

— 裁剪模式

- 词表裁剪：移除词表中未被使用的单词，实现减小模型体积，提升MLM任务速度的效果
- Transformer裁剪：裁剪“不重要”的注意力头和全连接层神经元，在减小模型体积的同时把对模型性能的影响尽可能降到最低
- 流水线裁剪：依次分别进行Transformer裁剪和词表裁剪，对模型体积做全面的压缩

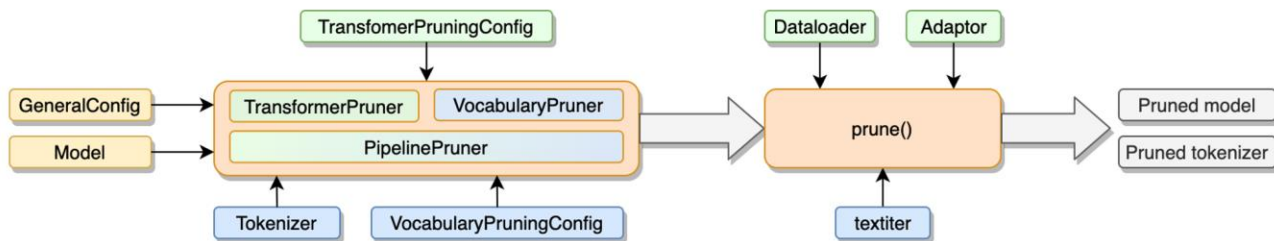




大语言模型的压缩

□ TextPruner

— 裁剪方法



Pruning with Python API

```
from textpruner import PipelinePruner
from textpruner import TransformerPruningConfig

config = TransformerPruningConfig(
    pruning_method='iterative', n_iters=4,
    target_ffn_size=2048, target_num_of_heads=8)
pruner = PipelinePruner(model, tokenizer, config)
pruner.prune(dataloader=dataloader, dataiter=texts)
```

▲ 通过Python API裁剪

Pruning with CLI

```
textpruner-cli \
--pruning_mode pipeline \
--configurations vc.json trm.json \
--model_class BertForClassification \
--tokenizer_class BertTokenizer \
--model_path models \
--vocabulary texts.txt \
--dataloader_and_adaptor dl.py
```

▲ 通过命令行裁剪



大语言模型的压缩

□ 参数量化

- 参数量化是通过降低模型权重和激活值的表示精度（如从 FP32 转换为 INT8 或更低）来减小模型尺寸与计算成本的压缩方法
- 相比知识蒸馏和模型裁剪，量化直接作用于模型数值表示，无需结构变更或额外训练模型，特别适用于边缘设备部署
- 两类主流方法
 - PTQ（训练后量化）
 - 优点：部署简便，不依赖训练数据。
 - 缺点：若量化精度较低，可能导致精度损失较大。
 - QAT（量化感知训练）
 - 优点：在训练阶段引入量化过程，可有效缓解精度下降。
 - 缺点：训练代价高，需样本和算力支持。

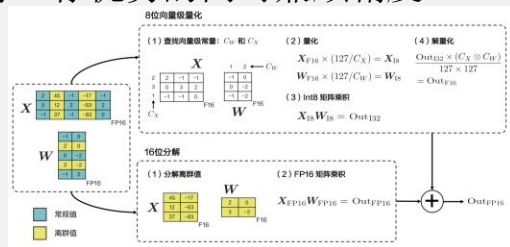




大语言模型的压缩

□ 参数量化: LLM.int8()

- LLM.int8() 是一种为大型 Transformer 模型设计的 8 位整数量化推理方法
- 在保持精度的同时将推理所需内存减少约一半，极大提升推理效率
- 两大核心技术
 - 向量级量化（Vector-wise Quantization）：
 - 利用 Absmax 技术将数据缩放至 $[-127, 127]$ ，每行（输入）与每列（权重）分别使用独立缩放因子（ $C_X \otimes C_W$ ），增强对离群值的鲁棒性。
 - 混合精度分解（Mixed-Precision Decomposition）
 - 针对高幅值的离群特征，采用 F16 计算，其余则保留为 Int8；仅约 0.1% 特征使用 F16，从而维持显存优势的同时兼顾精度。





大语言模型的压缩

□ 参数量化: GPTQ

- GPTQ 是一种高效的训练后量化（PTQ）方法，可将 GPT、OPT 等大型语言模型压缩至 3–4 位精度，在数小时内完成百亿级参数模型的量化，同时保持高精度
- 构建在OBQ（Optimal Brain Quantization）之上，并引入三项关键改进

1、任意顺序洞察

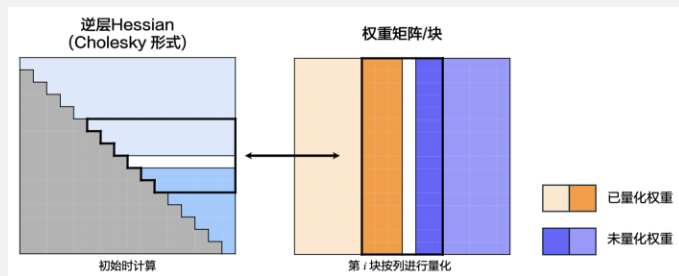
- 放宽传统贪婪顺序限制，所有行使用相同的列顺序；
- 大模型中即使初期引入较大误差，后期仍可修正；
- 显著减少更新次数，加速数十倍以上。

2、延迟批次更新

- 对每个 128 列的块整体处理，仅更新该块对应的 Hessian 子块；
- 减少内存访问，提高 GPU 吞吐效率；
- 对 W 和 H 的全局更新延后执行，减少瓶颈。

3、Cholesky 重构

- 为解决 Hessian 数值不稳定，引入 Cholesky 分解预计算；
- 只需处理特定行，提升精度稳定性；
- 同时加入对角阻尼因子，增强收敛与数值安全性。





小结

- 基于提示的推断
- 多任务指令微调
- 基于人类反馈的强化学习
- 参数高效精调
- 大语言模型的中文适配
- 大语言模型压缩

